DevSecOps

Автоматизация тестирования безопасности

Вспомним

- Чем exploit отличается от payload?
- Что такое ZeroDay уязвимость?
- Что в оценке CVSS означает метрика
  Exploit Maturity (доступность средств эксплуатации): POC ?
- Зачем нужен каталог KEV?

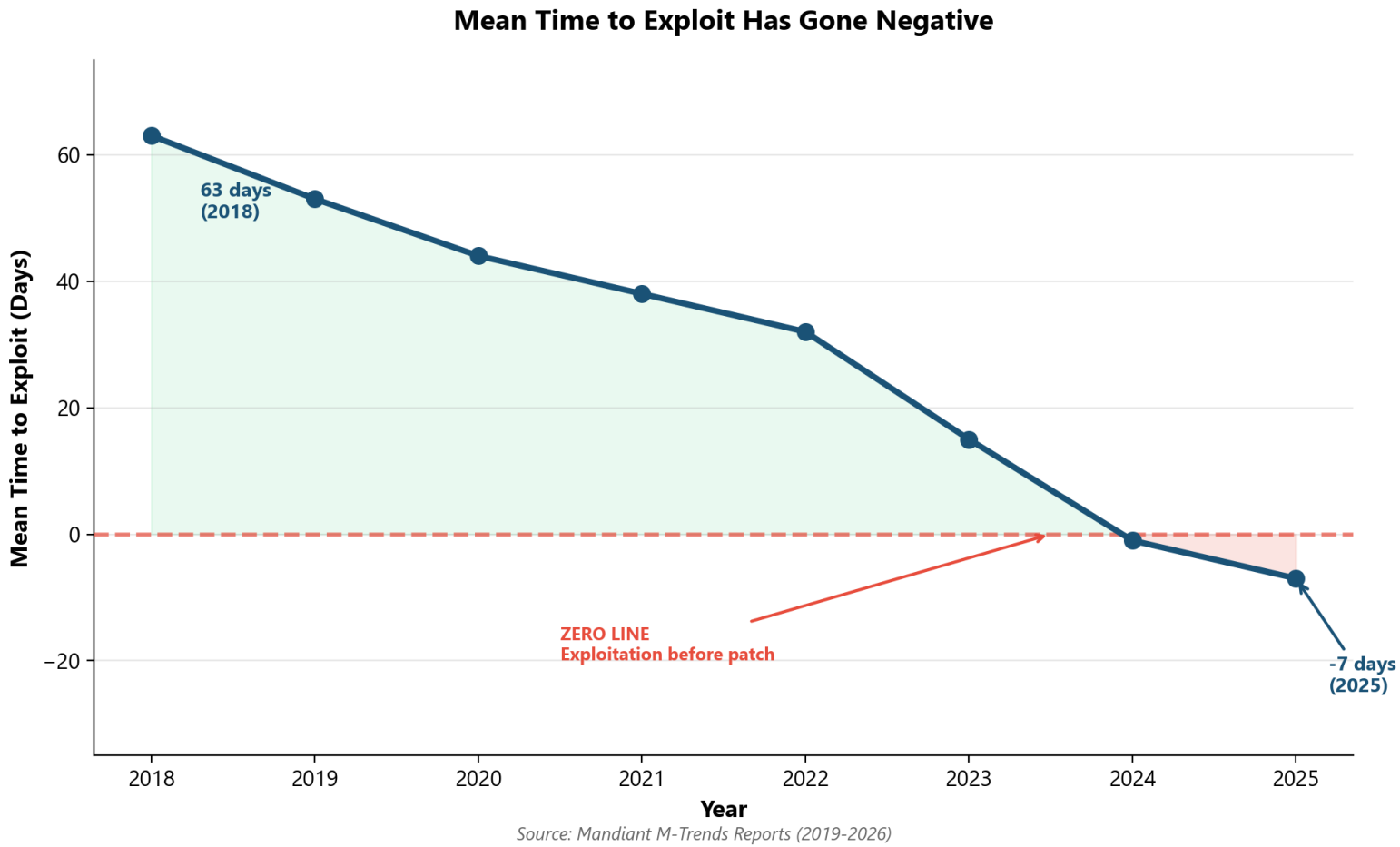
План лекции

- Окно уязвимости - скорость эксплуатации
- Принципы DevSecOps
- Инструменты SAST, DAST, SCA
- Встраивание в CI/CD
- Работа с находкой

Скорость эксплуатации уязвимостей

Почему управление уязвимостями требует автоматизации

Time-to-exploit

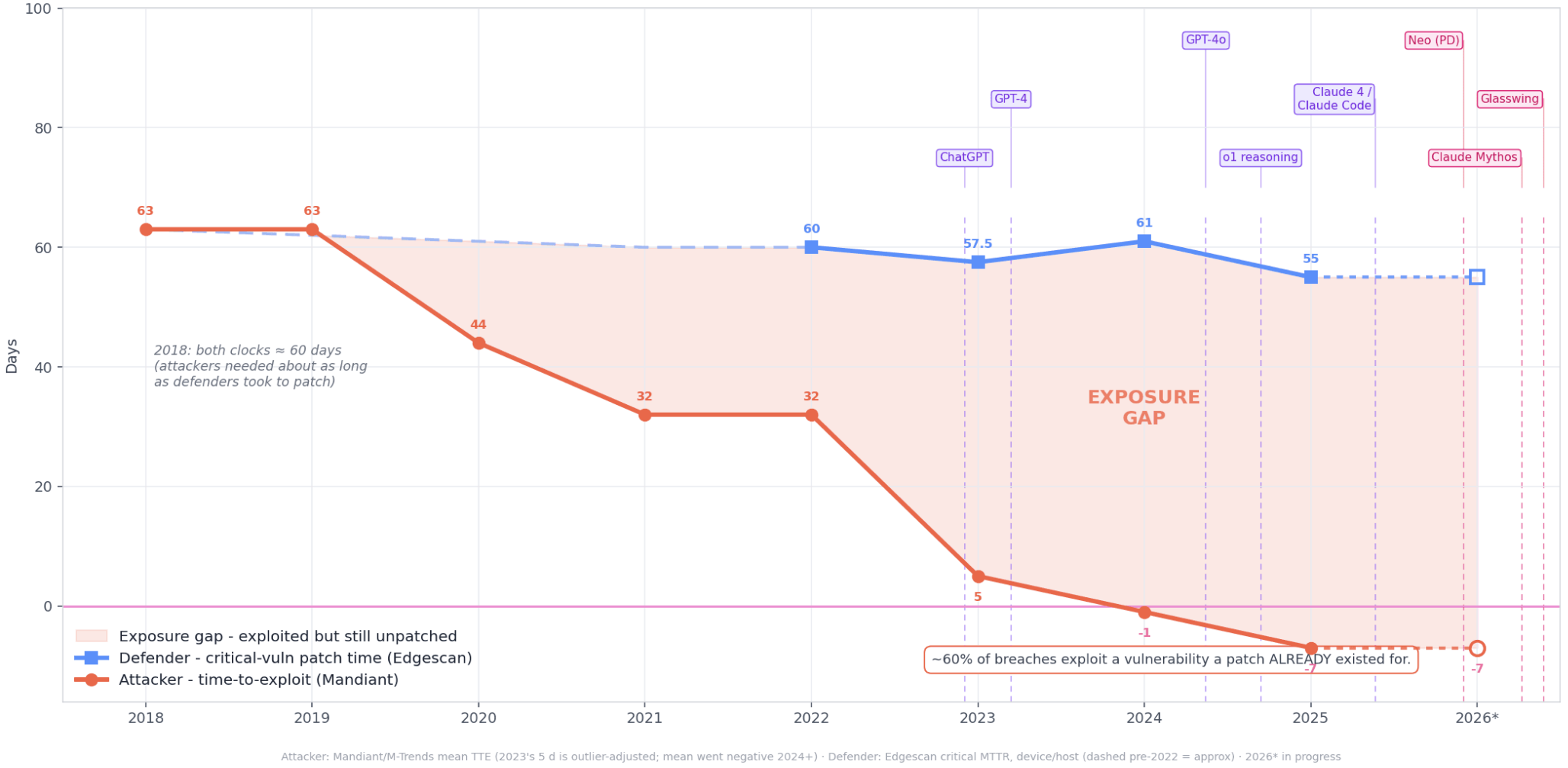


Source: Mandiant M-Trends Reports (2019-2026)

TTE = дата первой зафиксированной эксплуатации − дата раскрытия уязвимости

Источник: Mandiant M-Trends reports 2019-2026

Разрыв во времени патчинга



Источник: https://projectdiscovery.io/blog/the-vulnerability-curve-bent-with-the-ai-curve

Термины

- **Zero-day (уязвимость нулевого дня)**
  - уязвимость, о которой разработчик еще не знает или для которой еще нет исправления
- **N-day (уязвимость N-го дня)**
  - уязвимость, которая уже публично раскрыта и для которой вендор выпустил исправление, но установлено оно пока лишь на части систем
- **N-hour** - ?

Ускорение эксплуатации уязвимостей

120 -> 80 дней

медианное время от публикации CVE до внесения в KEV

2025 -> H1 2026 [1]

28.96% KEVs

эксплуатируются в день публикации CVE или раньше

H1 2026 [1]

< 1 часа

требуется ИИ для написания эксплоита к N-day

Anthropic Mythos, 2026 [2]

1 - https://www.servnetuk.com/research/cve-exploitation-speed-index-2026
2 - https://www.anthropic.com/research/n-days

Vulnerability Pressure Index

Quarterly comparison with the same three months a year earlier, across the three metrics that matter most.

Vulnerability pressure

New CVE records published this quarter, against the same quarter last year

270 % of last year **Double**

30k CVE records published in 3 months, against **11k** in the same three months last year.

Scanning pressure

Traffic for the 100 most-probed vulnerabilities this quarter, against the same last year

52 % of last year **Decline**

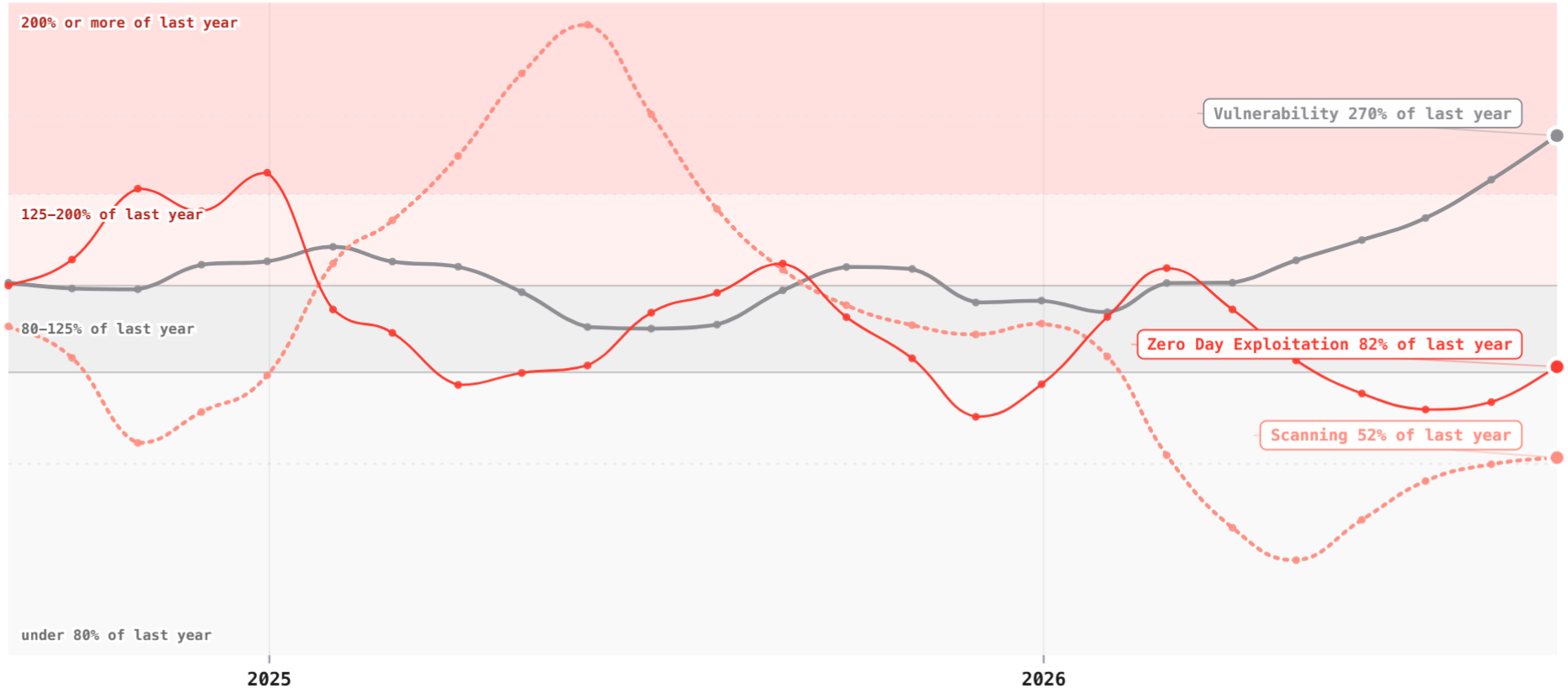
305k daily scanning rate, top 100 summed in 3 months, against **591k** in the same three months last year.

Zero Day Exploitation pressure

CVEs exploited (KEV) when they went public, against the same quarter last year

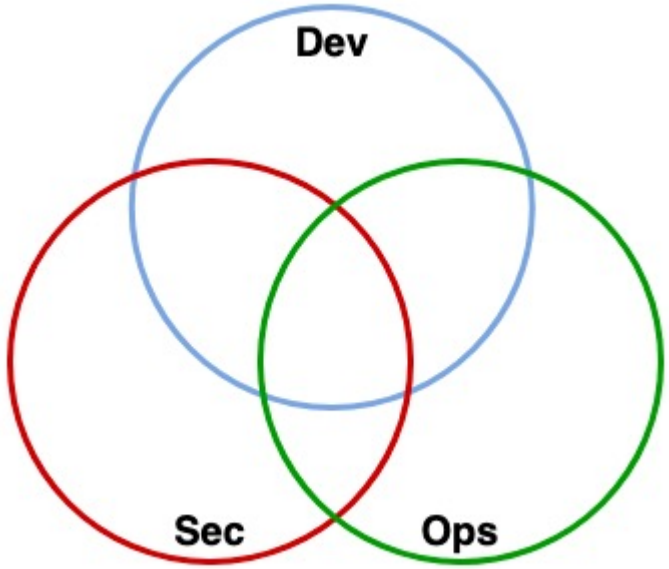
82 % of last year **Similar**

98 listed no later than published in 3 months, against **119** in the same three months last year.



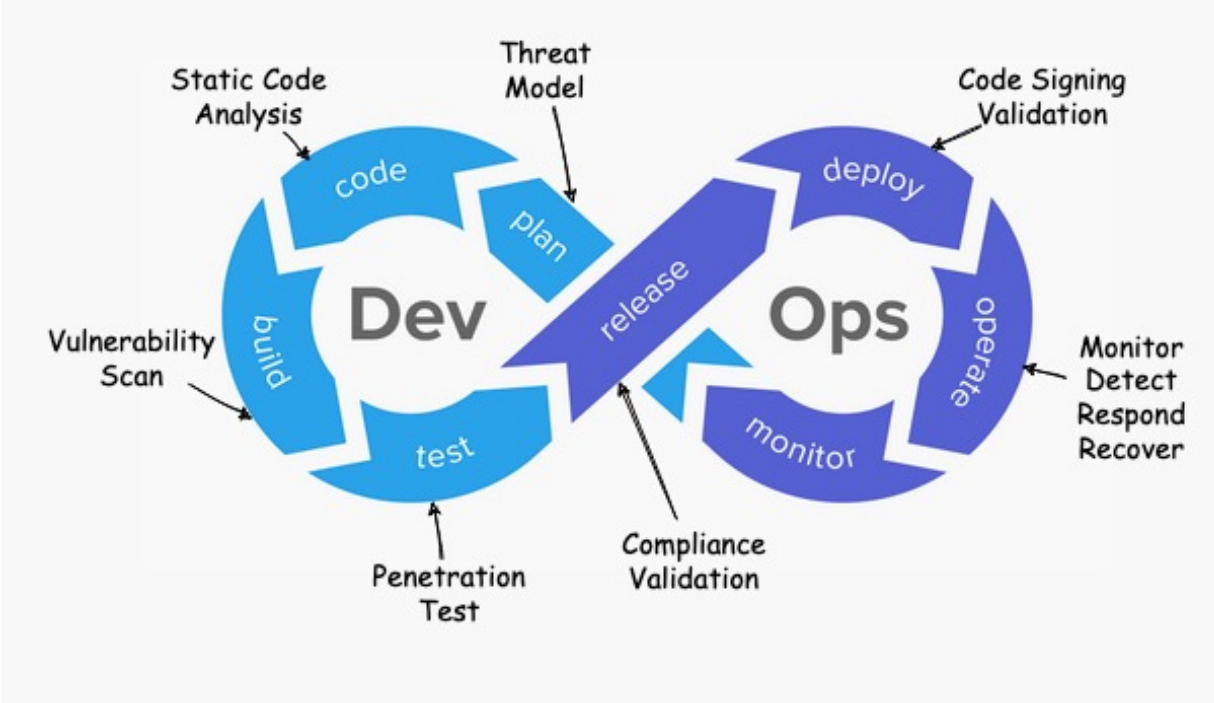
Source: CVE Project · NVD · CISA KEV · EUVD KEV · CIRCL KEV · Previdian KEV · VulnCheck KEV · Shadowserver

DevSecOps



DevSecOps

Направление деятельности по встраиванию процессов и инструментов безопасности в CI/CD для автоматизации выявления и устранения уязвимостей на всех этапах цикла разработки



Принципы DevSecOps



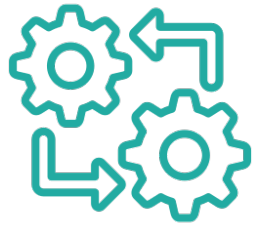
Shift Left

встраивание
в процесс SDLC
с самого начала



Совместная ответственность

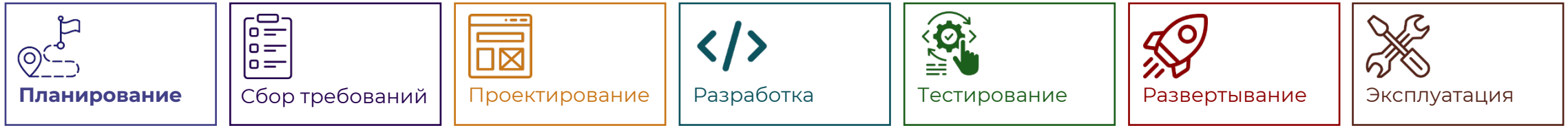
Dev + Sec + Ops
+ AI-агенты



Автоматизация

от обнаружения
до исправления
без ручных шагов

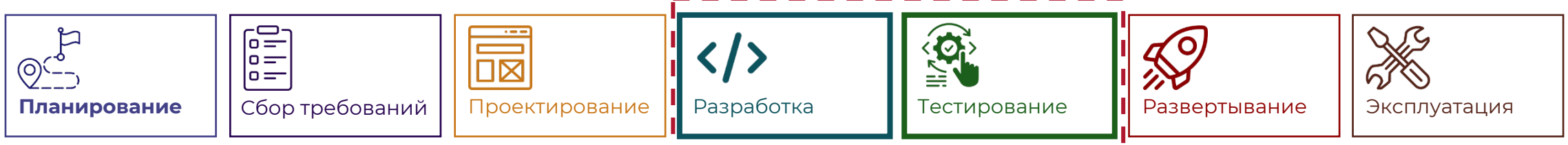
SDLC



ИИ – агенты на всех этапах

SDLC

Фокус лекции



ИИ – агенты на всех этапах

Автоматизированное тестирование приложения

Методы и инструменты

SAST
- Static Application Security Testing

Метод анализа исходного кода, байт-кода
или бинарных файлов без запуска приложения
с целью обнаружения потенциальных уязвимостей

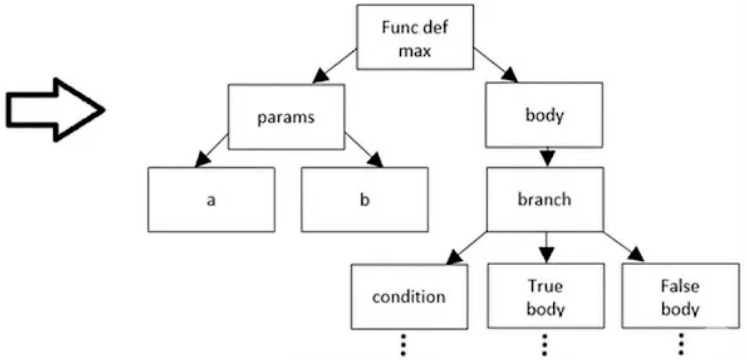
Принцип:

- Принимает на вход исходный код
- Проверяет его по определенному набору правил
- Может выдавать ложно-положительные находки

AST

Абстрактное синтаксическое дерево - древовидное представление исходного кода, показывающее структуру программы

```
int max(int a, int b) {
    if (a > b) {
        return a;
    }
    else {
        return b;
    }
}
```



Что делает SAST:

1. Парсинг кода и построение AST
2. Обход дерева и поиск по паттернам
3. Выявление опасных паттернов

Правило semgrep – метаданные

```yaml
rules:
- id: tainted-file-path
  languages:
  - java
  severity: ERROR
  message: >-
    Detected user input controlling a file path. An attacker could control the location of this
    file, to include going backwards in the directory with '../'. To address this, ensure that user-controlled
    variables in file paths are sanitized. You may also consider using a utility method such as org.apache.commons.io.FilenameUtils.getName(...)
    to only retrieve the file name from the path.
  options:
    interfile: true  # анализ идёт не в рамках одного файла, а через весь проект
  metadata:
    cwe:
    - 'CWE-23: Relative Path Traversal'
    owasp:
    - A01:2021 - Broken Access Control
    - A01:2025 - Broken Access Control
    references:
    - https://owasp.org/www-community/attacks/Path_Traversal
    category: security
    technology:
    - java
    - spring
```

Правило semgrep – taint анализ

```
31      mode: taint  # включает отслеживание потока данных
32      pattern-sources:  # источник недоверенных данных
33      - patterns:
34        - pattern-either: # OR
35          - pattern-inside: |
36              $METHODNAME(..., @$REQ(...) $TYPE $SOURCE,...) {
37                ...
38              }
39          - pattern-inside: |
40              $METHODNAME(..., @$REQ $TYPE $SOURCE,...) {
41                ...
42              }
43        - metavariable-regex:
44            metavariable: $TYPE
45            regex: ^(?!(Integer|Long|Float|Double|Char|Boolean|int|lo
46        - metavariable-regex:
47            metavariable: $REQ
48            regex: (RequestBody|PathVariable|RequestParam|RequestHead
49        - focus-metavariable: $SOURCE
50      pattern-sinks:  # опасный интерпретатор
51      - patterns:
52        - pattern-either:
53          - pattern: new File(...)
54          - pattern: new java.io.File(...)
55          - pattern: new FileReader(...)
56          - pattern: new java.io.FileReader(...)
57          - pattern: new FileInputStream(...)
58          - pattern: new java.io.FileInputStream(...)
59          - pattern: (Paths $PATHS).get(...)
60          - patterns:
61            - pattern: |
62                $CLASS.$FUNC(...)
63            - metavariable-regex:
64                metavariable: $FUNC
65                regex: ^(getResourceAsStream|getResource)$
66          - patterns:
67            - pattern-either:
68              - pattern: new ClassPathResource($FILE, ...)
69              - pattern: ResourceUtils.getFile($FILE, ...)
70              - pattern: new FileOutputStream($FILE, ...)
71              - pattern: new java.io.FileOutputStream($FILE, ...)
72              - pattern: new StreamSource($FILE, ...)
73              - pattern: new javax.xml.transform.StreamSource($FILE, ...)
74              - pattern: FileUtils.openOutputStream($FILE, ...)
75            - focus-metavariable: $FILE
76      pattern-sanitizers:  # безопасный санитайзер
77      - pattern: org.apache.commons.io.FilenameUtils.getName(...)
```

structure NEW | advanced

search | taint

Sources

- & all
  - || any
    - inside
      - </> pattern

```
$METHODNAME(..., @$REQ(...) $TYPE $SOURCE,..
  ...
}
```

    - inside
      - </> pattern

```
$METHODNAME(..., @$REQ $TYPE $SOURCE,...) {
  ...
}
```

Rule info

Rule ID

tainted-file-path

Language

Java

Severity

ERROR

Message

Detected user input controlling a file path. An attacker could control the location of this file, to include going backwards in the directory with '../'. To address this, ensure that user-controlled

test code | live code NEW | metadata | docs

```
52  public static void bad(@RequestParam String user)
53  {
54      Socket sock;
55      BufferedReader filenameReader = new BufferedReader(
56              new InputStreamReader(sock.getInputStream(), "UTF-8"));
57      String filename = filenameReader.readLine();
58      // ruleid: tainted-file-path
59      BufferedReader fileReader = new BufferedReader(new FileReader("/home/
        " + user + "/" + filename));
60      String fileLine = fileReader.readLine();
61      while(fileLine != null) {
62          sock.getOutputStream().write(fileLine.getBytes());
63          fileLine = fileReader.readLine();
64      }
65  }
66
67  public static void bad2(@RequestParam String filename)
68  {
69      ApplicationContext appContext =
70        new ClassPathXmlApplicationContext(new String[] {"If-you-have-any.
          xml"});
71
72      // ruleid: tainted-file-path
73      Resource resource = appContext.getResource("classpath:com/" +
        filename);
74
75      try {
76        InputStream is = resource.getInputStream();
77        BufferedReader br = new BufferedReader(new InputStreamReader(is));
78
79        String line;
80        while ((line = br.readLine()) != null) {
81          System.out.println(line);
82        }
83        br.close();
84
85      } catch(IOException e){
86        e.printStackTrace();
87      }
```

Run Ctrl-⏎

4 matches

Semgrep v1.171.0 · in 1.0s · tests passed

Особенности SAST

Хорошее качество анализа

- Инъекции (SQL injection, Command injection, XXE...)
- Криптография

Плохое качество анализа

- Уязвимости бизнес-логики
- Нарушения контроля доступа (авторизация, IDOR)

Особенности SAST

- Ложноположительные и ложнонегативные находки
  - Из-за ограниченной глубины анализа потока данных, кастомных санитайзеров и особенностей контекста приложения
  - Средний recall ~ 20%, precision ~ 50-90% [1]
- Необходимость сборки исходного кода
- Ограниченность правилами
- Зависимость от языка

1 - https://dl.acm.org/doi/10.1145/3727967.3756836

SAST инструменты

1. Semgrep Code
2. CodeQL
3. SonarQube
4. Solar appScreener
5. PT Application Inspector
6. PVS-Studio

...

1. https://github.com/semgrep/semgrep
2. https://github.com/github/codeql
3. https://www.sonarsource.com/products/sonarqube/
4. https://rt-solar.ru/products/solar_appscreener
5. https://ptsecurity.com/products/ai/
6. https://pvs-studio.ru/

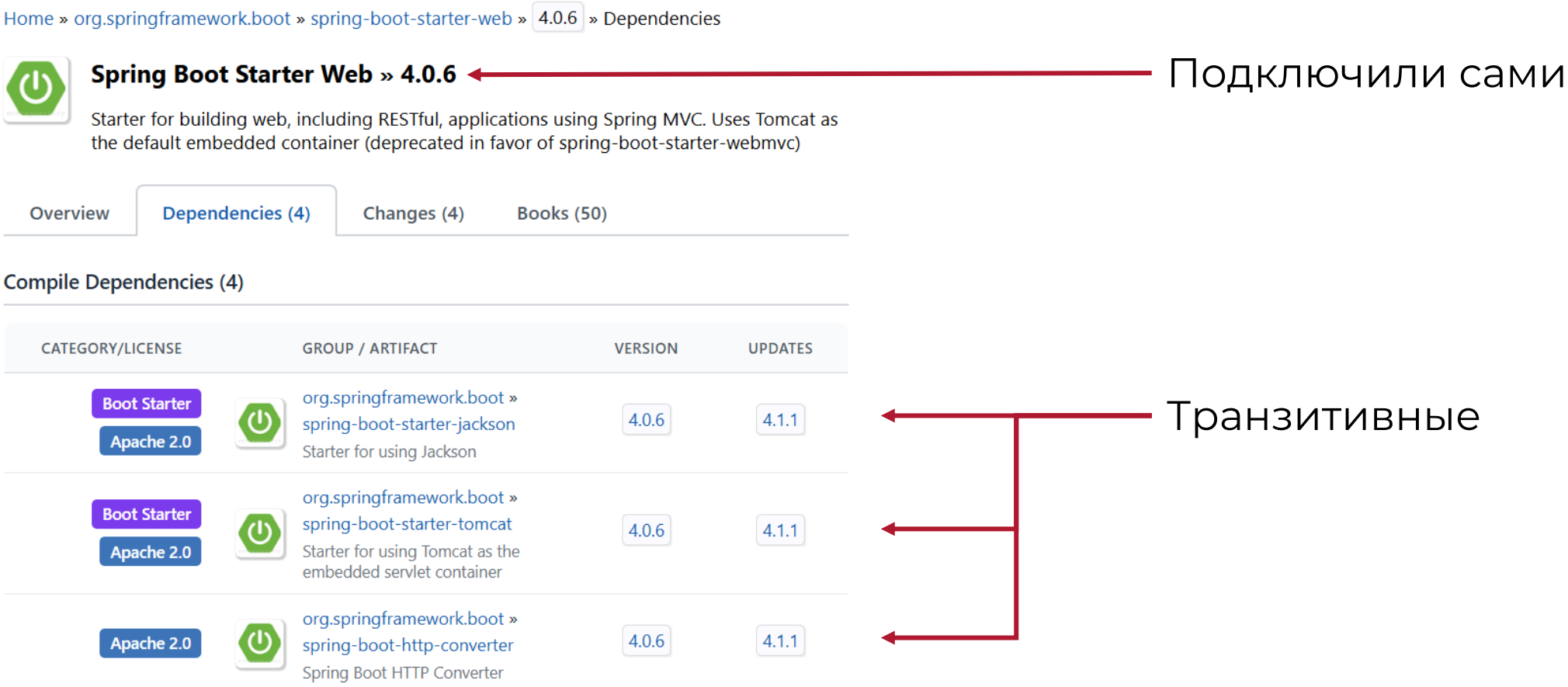
SCA - Software Composition Analysis

Метод анализа сторонних и открытых компонентов ПО
с целью выявления известных уязвимостей, устаревших версий

Принцип:

1. Инструмент строит полный список используемых компонентов приложения, включая **транзитивные** зависимости
2. Каждый компонент сопоставляется с базами данных известных уязвимостей по имени и версии
3. Находка привязывается к конкретному CVE-идентификатору, оценке критичности (CVSS), EPSS и другим метрикам

Транзитивная зависимость



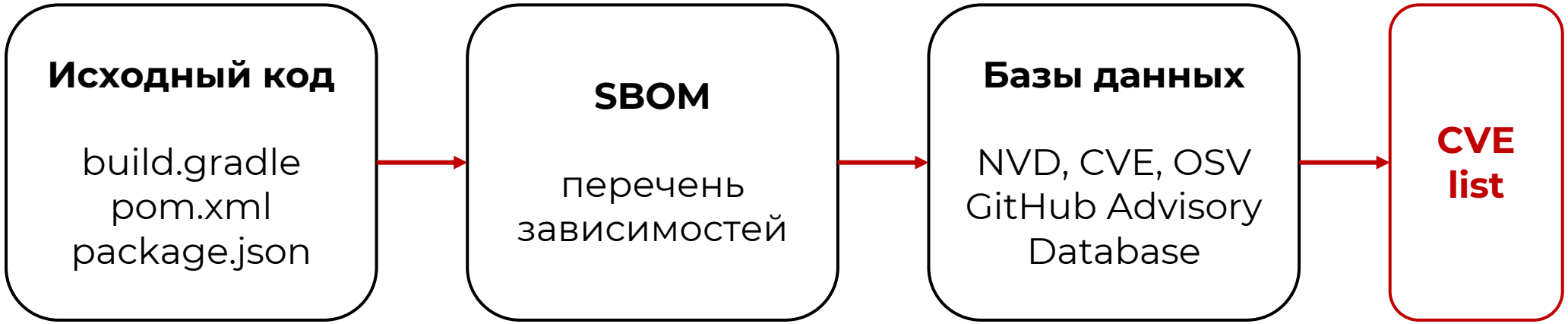
SBOM

Software Bill of Materials - структурированный, машиночитаемый перечень всех компонентов, из которых состоит приложение

Форматы: SPDX, CycloneDX

```
"dependencies": [
        {
            // Основной компонент и его прямые зависимости
            "ref": "pkg:maven/com.appsecco/dvja@1.0-SNAPSHOT?type=war",
            "dependsOn": [
                "pkg:maven/cglib/cglib@2.2?type=jar",
                "pkg:maven/com.google.code.gson/gson@2.8.1?type=jar",
                "pkg:maven/com.jgeppert.struts2.bootstrap/struts2-bootstrap-plug
                "pkg:maven/commons-codec/commons-codec@1.10?type=jar",
                "pkg:maven/javax.servlet/jsp-api@2.0?type=jar",
                "pkg:maven/javax.servlet/servlet-api@2.4?type=jar",
                "pkg:maven/junit/junit@4.5?type=jar",
                "pkg:maven/mysql/mysql-connector-java@5.1.42?type=jar",
                "pkg:maven/org.apache.logging.log4j/log4j-api@2.3?type=jar",
                "pkg:maven/org.apache.logging.log4j/log4j-core@2.3?type=jar",
                "pkg:maven/org.apache.struts/struts2-config-browser-plugin@2.3.30
                "pkg:maven/org.apache.struts/struts2-core@2.3.30?type=jar",
                "pkg:maven/org.apache.struts/struts2-junit-plugin@2.3.30?type=ja
                "pkg:maven/org.apache.struts/struts2-spring-plugin@2.3.30?type=j
                "pkg:maven/org.hibernate/hibernate-annotations@3.4.0.GA?type=jar
                "pkg:maven/org.hibernate/hibernate-core@3.3.1.GA?type=jar",
                "pkg:maven/org.hibernate/hibernate-entitymanager@3.4.0.GA?type=j
                "pkg:maven/org.slf4j/slf4j-log4j12@1.5.2?type=jar",
                "pkg:maven/org.springframework/spring-orm@3.0.5.RELEASE?type=jar
                "pkg:maven/org.springframework/spring-tx@3.0.5.RELEASE?type=jar"
            ]
        },
```

SCA – принцип работы



Особенности SCA

- Проблема достижимости

  Наличие уязвимой версии библиотеки в дереве зависимостей не означает, что приложение вызывает уязвимый код внутри нее. FP ~ 60%.
- Зависимость от актуальности баз данных

  0day вне поля зрения
- Только известные CVE

  Другие уязвимости в коде – не зона ответственности SCA

SCA инструменты

1. Trivy
2. OWASP Dependency-Check
3. OWASP Dependency-Track
4. Snyk

...

1. https://trivy.dev
2. https://owasp.org/www-project-dependency-check/
3. https://dependencytrack.org
4. https://snyk.io

DAST
- Dynamic Application Security Testing

Метод анализа безопасности работающего приложения путем взаимодействия с ним и анализа его поведения
без доступа к исходному коду

Подвиды DAST

- Поиск известных CVE
- Анализ на основе правил
- Фаззинг

Особенности DAST

- Языковая независимость
- Хорошо находит внешне видимые проблемы
- Проверяет окружение - конфигурацию сервера, окружение развертывания, взаимодействие компонентов в реальном времени
- Ограниченное покрытие
- Поздний этап обнаружения
- Требует тестовый стенд и сетевой доступ
- Генерирует нагрузку

```yaml
id: tomcat-stacktraces

info:
  name: Tomcat Stack Traces Enabled
  author: lucky0x0d
  severity: low
  description: |
    Examine whether Tomcat stack traces are turned on by employing a designated problematic pattern.
  classification:
    cpe: cpe:2.3:a:apache:tomcat:*:*:*:*:*:*:*:*
    cwe-id: CWE-209
  metadata:
    verified: true
    max-request: 1
    vendor: apache
    product: tomcat
    shodan-query: title:"Apache Tomcat"
  tags: misconfig,tech,tomcat,apache,vuln

http:
  - method: GET
    path:
      - '{{BaseURL}}/?f=\['

    matchers:
      - type: dsl
        dsl:
          - 'contains(body, "tomcat")'
          - 'contains(body, "org.apache")'
          - status_code == 400
        condition: and
```

Инструменты DAST

1. OWASP ZAP
2. Burp Suite
3. Nuclei

...

1. https://www.zaproxy.org
2. https://portswigger.net/burp
3. https://github.com/projectdiscovery/nuclei

Другие классы инструментов

- **Secrets scanning** - поиск чувствительных данных в коде
- **IAST** (Interactive Application Security Testing) - инструментирует приложение агентами, которые отслеживают его поведение
- **IaC scanning** - статический анализ файлов инфраструктуры как кода (Terraform, Helm charts, Dockerfile)
- **Container / image scanning** - сканирование образов на уязвимости в пакетах ОС, неверные конфигурации и вредоносный код

ИИ в тестировании безопасности

ИИ как помощник классических сканеров

- SAST: отсеивает FP
- SCA: проверяет достижимость
- DAST: помогает строить сценарии атак и подтверждать находки

ИИ как самостоятельный анализатор кода

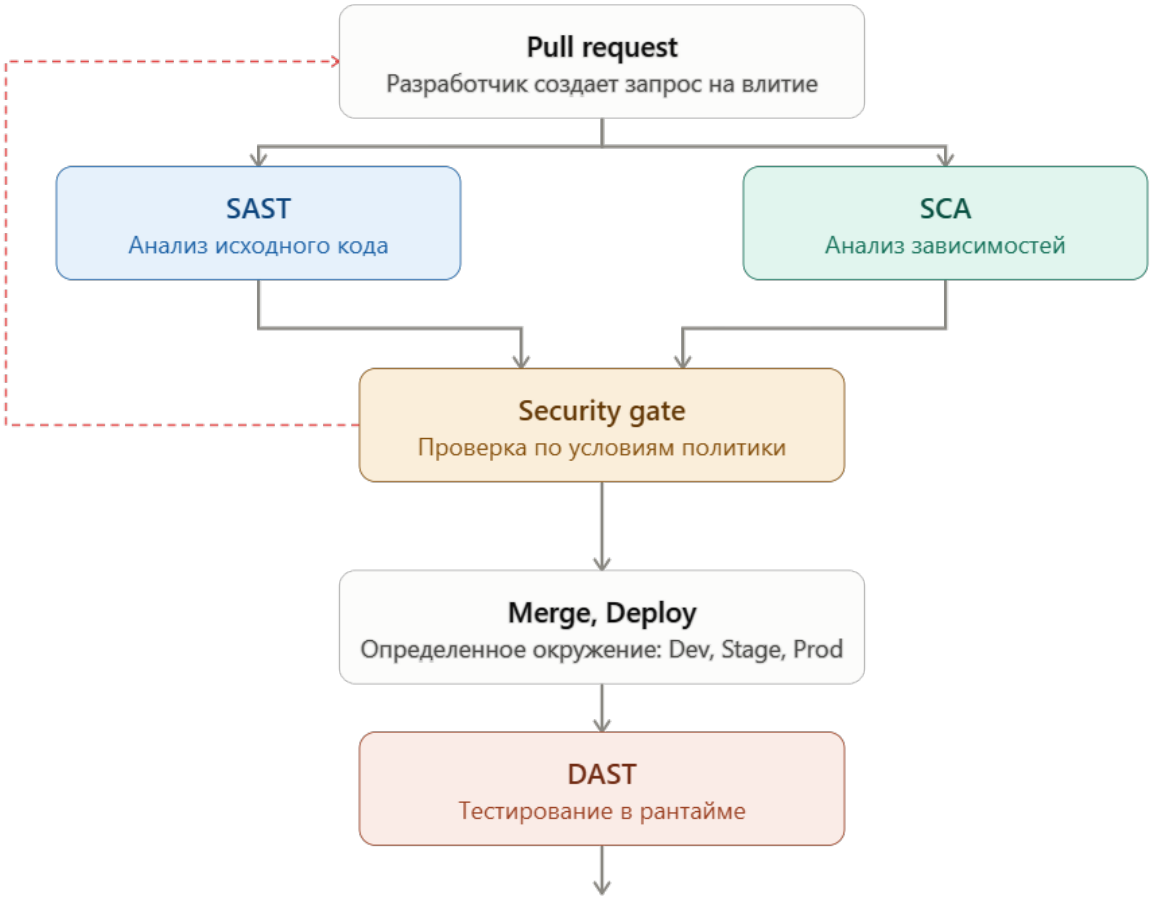
- LLM-ревью безопасности
- находит часть ошибок логики и контроля доступа
- результат требует проверки человеком

Встраивание сканеров в пайплайн

Security Gate

Этап в пайплайне, который проверяет находки сканеров по заданной политике и выносит вердикт:

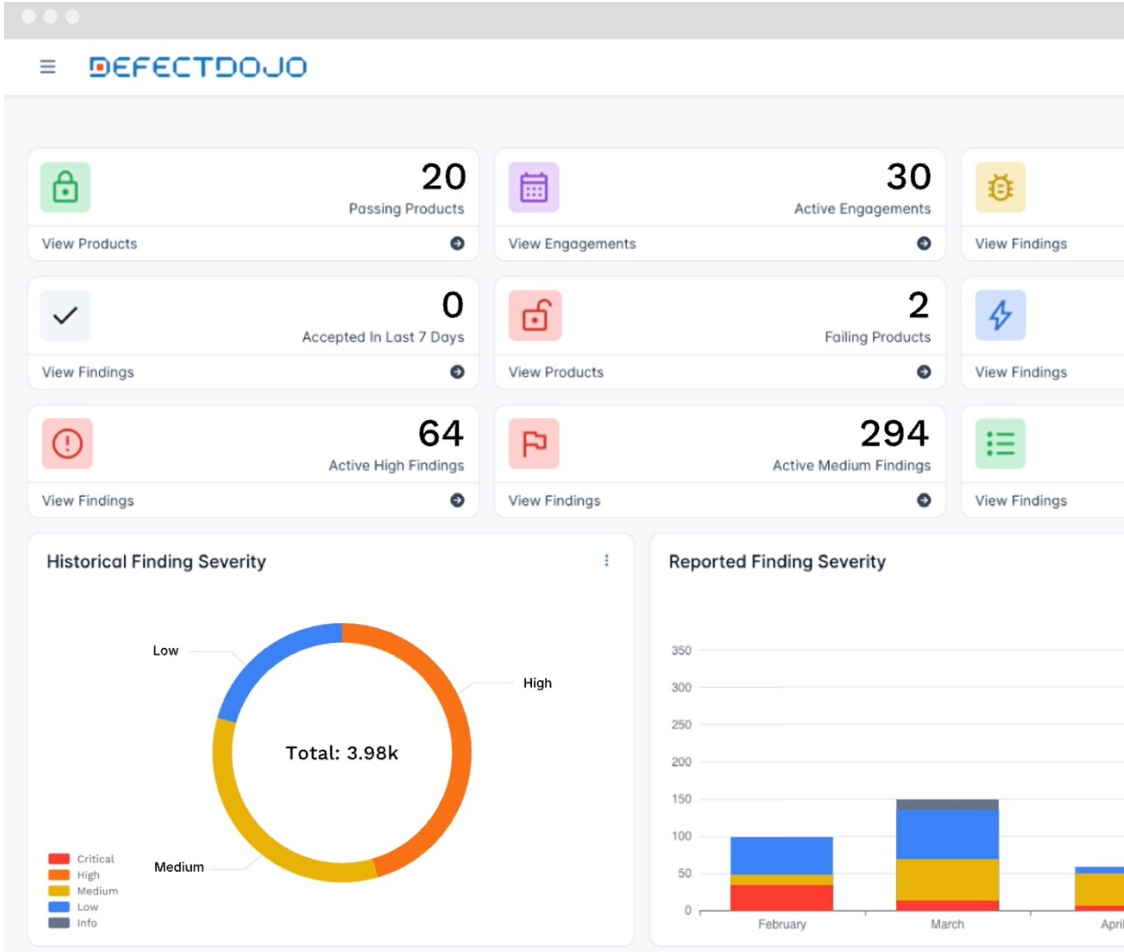
- пропустить сборку
- выдать предупреждение
- заблокировать

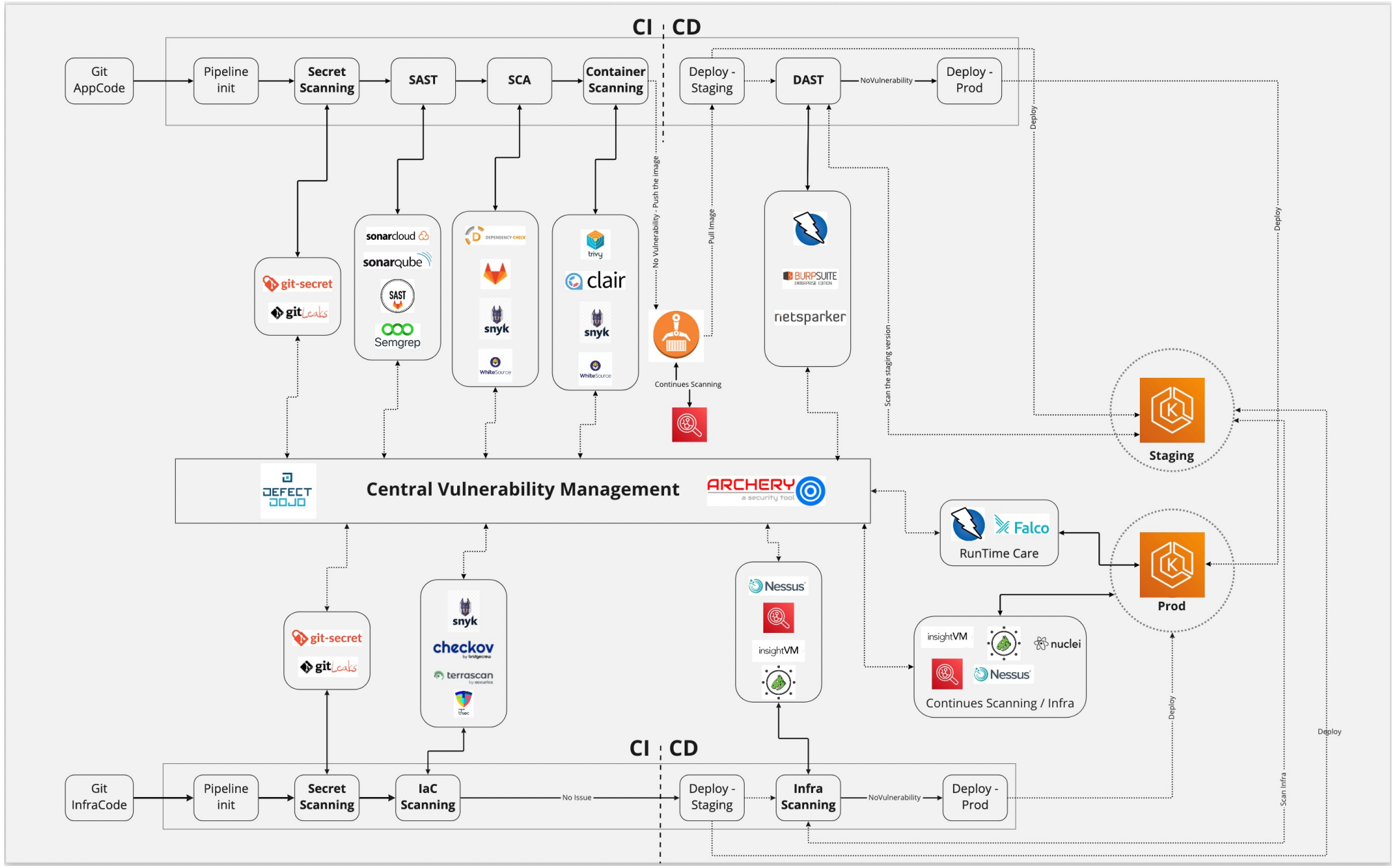


ASOC - Application Security Orchestration and Correlation

Категория инструментов, которые обеспечивают

- хранение и возможность обработки находок разных сканеров в едином месте
- корреляцию, нормализацию и дедупликацию находок
- координацию времени и способа запуска сканеров





Источник: OWASP DevSecOps Guideline - v-0.2

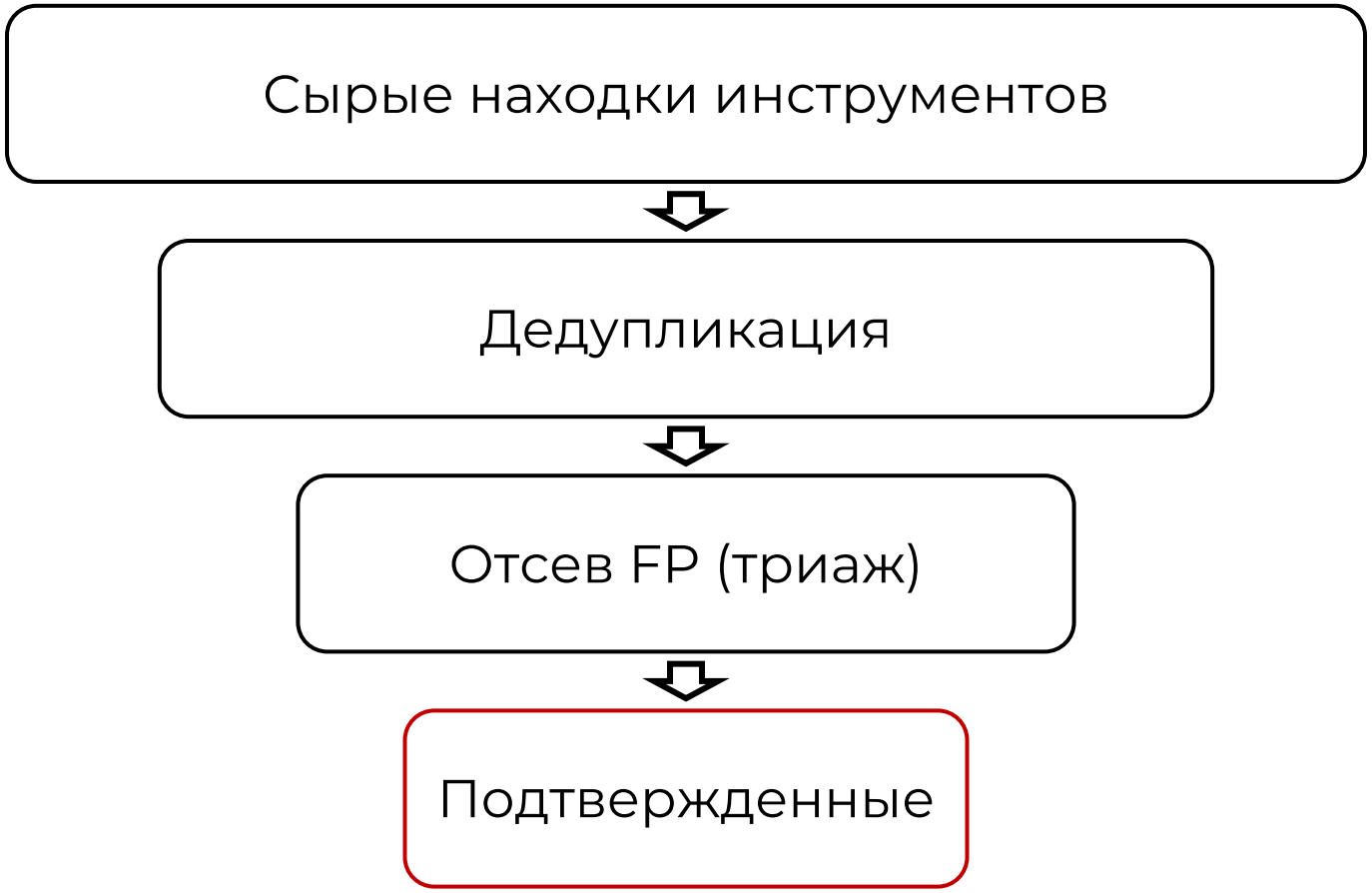
Каждый сканер видит свою часть рисков

	SAST	DAST	SCA
Инъекции	Сильный	Сильный	Не видит
Криптография, секреты	Сильный	Частично	Не видит
Уязвимые библиотеки	Не видит	Частично	Сильный
Конфигурация сервера	Частично	Сильный	Не видит
Доступ и бизнес-логика	Слабо	Частично	Не видит

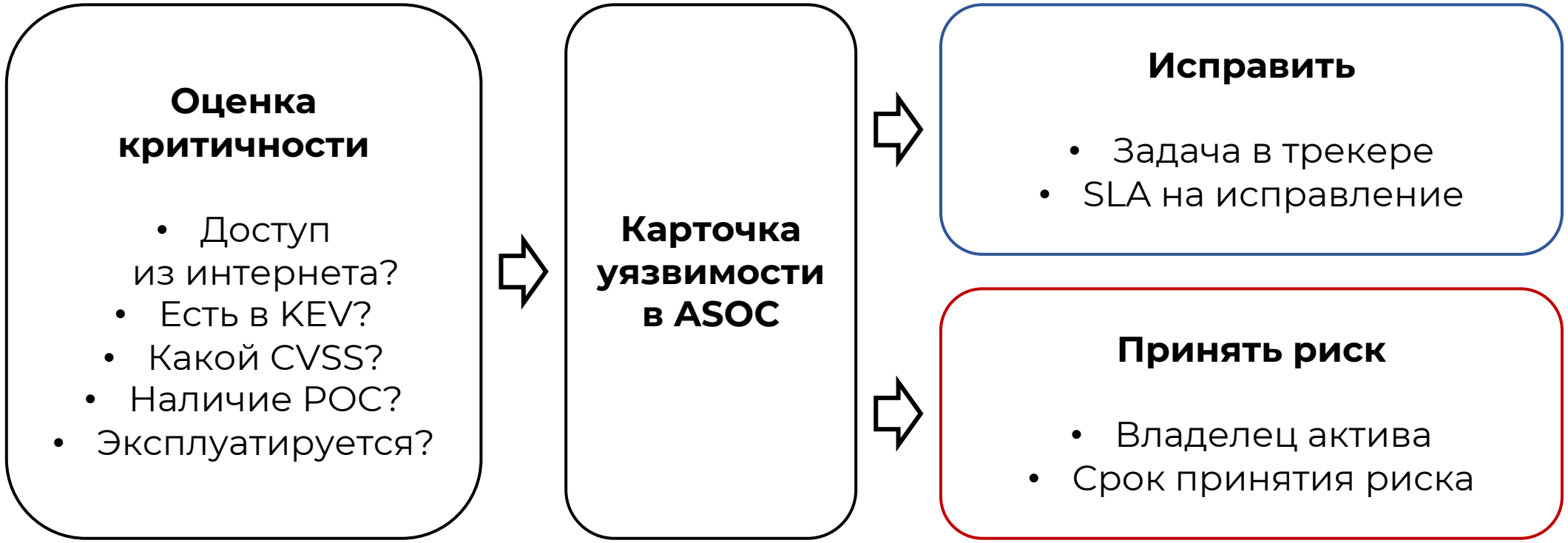
Работа с находкой

Триаж, работа с FP, метрики

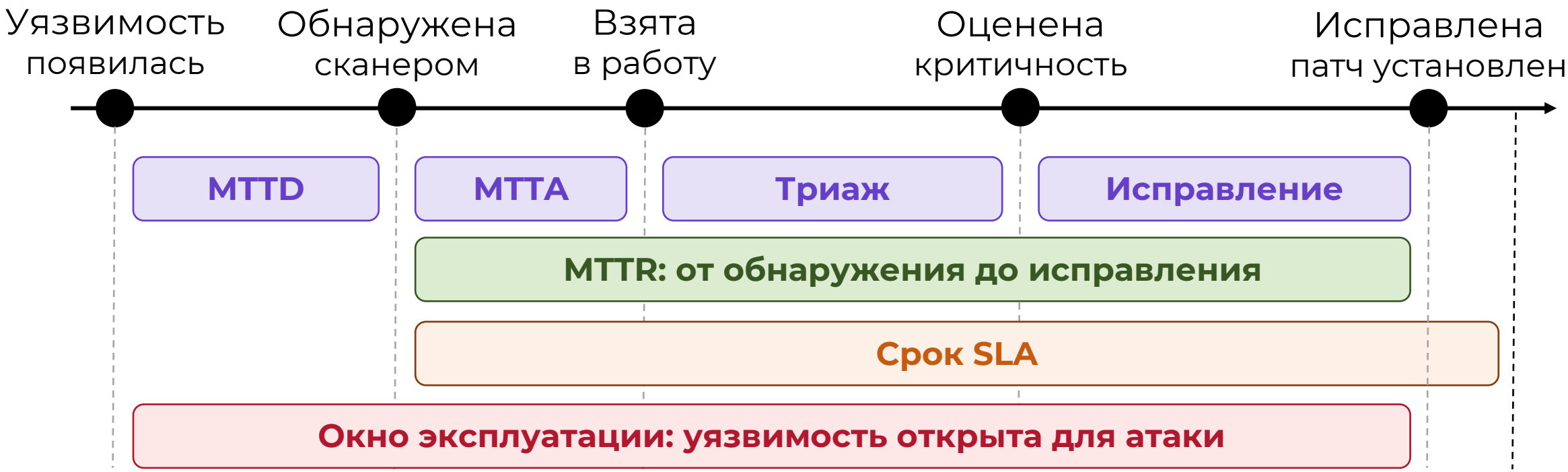
Воронка находок



Приоритизация и вердикт



Метрики – скорость реакции



- **MTTR** (Mean Time to Remediate) - среднее время от обнаружения уязвимости до исправления
- **Соблюдение SLA** (Service Level Agreement) - доля уязвимостей, исправленных в установленный срок для своего уровня риска

Метрики – качество и охват

- **Покрытие** - доля репозиториев и конвейеров, где подключены SAST, SCA и DAST, и доля активов в инвентаре
- **Доля ложных срабатываний** - сколько находок после триажа оказались FP
- **Повторяемость** - как часто одна и та же проблема возвращается после исправления

Выводы

- Время между публикацией уязвимости и ее эксплуатацией сократилось до часов
- Автоматизация необходима на всем пути
  - от обнаружения уязвимости до исправления
- Каждый тип сканера находит свою часть уязвимостей
- Ошибки бизнес-логики сканеры почти не находят
- ИИ ускоряет триаж, помогает анализировать код и выявлять ошибки бизнес-логики, но требует проверки человеком

Ссылки

- https://projectdiscovery.io/blog/the-vulnerability-curve-bent-with-the-ai-curve
  - аналитическая статья о влиянии LLM на количество CVE
- https://www.anthropic.com/research/n-days
  - оценка скорости реализации эксплойтов LLM
- https://zerodayclock.com/
  - визуализация статистики по публично раскрытым уязвимостям
- https://dl.acm.org/doi/10.1145/3727967.3756836
  - оценка эффективности инструментов SAST
- https://owasp.org/www-project-devsecops-guideline/
  - обзор этапов и инструментов DevSecOps-конвейера

Вопросы